

Deep Dive — Optimization & Generalization

Part 01 —  Machine Learning Foundations · 01-ml-foundations/notes/02-optimization-and-generalization.md

The two questions that define ML: "can I fit this?" (optimization) and "will it hold up?" (generalization).

Part I — Optimization

1. Gradient descent from first principles

We want $\theta^* = \operatorname{argmin} J(\theta)$. The gradient $\nabla J(\theta)$ points in the direction of steepest *increase*, so step against it:

$$\theta \leftarrow \theta - \eta \nabla J(\theta)$$

η is the **learning rate**, the single most important hyperparameter in all of deep learning.

- Too small $\rightarrow$ crawls, may stall in a plateau.
- Too large $\rightarrow$ oscillates or diverges (loss $\rightarrow$ NaN).
- Just right $\rightarrow$ smooth decrease, then a plateau you attack with a schedule.

2. Worked derivation: linear regression

With $\hat{y}_i = w^T x_i + b$ and $J = (1/n) \sum (\hat{y}_i - y_i)^2$:

$$\begin{aligned} \partial J / \partial w &= (2/n) \sum (\hat{y}_i - y_i) x_i \\ \partial J / \partial b &= (2/n) \sum (\hat{y}_i - y_i) \end{aligned}$$

Implement exactly this in `../code/linear_models_from_scratch.py` and watch the loss curve. Logistic regression has an almost identical gradient — with cross-entropy loss and a sigmoid output, $\partial J / \partial w = (1/n) \sum (\sigma(w^T x_i + b) - y_i) x_i$. The sigmoid derivative cancels beautifully; that cancellation is *why* we pair cross-entropy with sigmoid/softmax rather than MSE.

3. Batch, stochastic, and mini-batch

Variant	Gradient computed on	Pros	Cons
Batch GD	all n examples	stable, exact	one slow step; won't fit in memory
SGD	1 example	fast, escapes shallow minima	very noisy
Mini-batch	32–8192 examples	GPU-friendly, good noise level	batch size is a hyperparameter

Mini-batch won. Batch size interacts with learning rate: the **linear scaling rule** says if you multiply batch size by k , multiply the learning rate by roughly k (with warmup).

4. Optimizers you'll meet

- **Momentum** — accumulate a velocity $v \leftarrow \beta v + \nabla J$, step with v . Dampens oscillation across ravines.
- **RMSProp** — divide by a running RMS of gradients; per-parameter adaptive step sizes.
- **Adam** — momentum + RMSProp + bias correction. The default for neural nets. $\beta_1=0.9$, $\beta_2=0.999$, $\epsilon=1e-8$.
- **AdamW** — Adam with *decoupled* weight decay. **This is what trains modern LLMs.** The fix matters: L2-in-the-loss and weight decay are not equivalent once you use adaptive step sizes.

5. Learning-rate schedules

- **Warmup** — start near zero and ramp up over the first few hundred/thousand steps. Prevents early divergence when Adam's second-moment estimates are still garbage. Mandatory for transformers.
- **Cosine decay** — smooth decay to ~ 0 over training. The modern default.
- **Step decay / ReduceLROnPlateau** — the classical options.
- The canonical LLM recipe: *linear warmup* $\rightarrow$ *cosine decay to 10% of peak*.

6. Reading loss curves

Symptom	Likely cause	Fix
Loss $\rightarrow$ NaN	LR too high, bad init, exploding gradients	lower LR, clip gradients, check for <code>log(0)</code>
Loss flat from step 0	LR too low, no signal, broken labels	raise LR, overfit 10 examples as a smoke test
Train $\downarrow$, val $\uparrow$	overfitting	regularise, more data, early stop
Train and val both high	underfitting / too much regularisation	bigger model, train longer, fewer constraints
Spiky val curve	batch too small, LR too high, small val set	larger batch, LR decay, bigger val set

The smoke test that saves days: before any real run, verify your model can *overfit a batch of 10 examples to $\sim$ zero loss*. If it can't, the bug is in your code, not your hyperparameters.

Part II — Generalization

7. The bias–variance decomposition

For squared error, expected test error decomposes as:

$$E[(y - \hat{f}(x))^2] = \underbrace{\text{Bias}[\hat{f}(x)]^2}_{\text{too simple}} + \underbrace{\text{Var}[\hat{f}(x)]}_{\text{too twitchy}} + \underbrace{\sigma^2}_{\text{irreducible}}$$

- **High bias** (underfit): wrong on train *and* test, consistently. Model too simple.
- **High variance** (overfit): great on train, poor on test; predictions swing with the training sample.
- **Irreducible noise**: your label quality ceiling. No model beats it — and mislabelled data is the most common reason a "good enough" model looks bad.

8. Regularization — the anti-variance toolkit

Technique	Mechanism	Where used
L2 / Ridge	penalise $\Sigma \theta^2$; shrinks weights smoothly	everywhere, incl. LLM training as weight decay
L1 / Lasso	penalise $\Sigma \theta $; drives weights to exactly 0 → feature selection	sparse tabular problems
Elastic Net	L1 + L2	correlated features
Early stopping	halt when val loss stops improving	universal, free
Dropout	randomly zero activations at train time	Module 02
Data augmentation	enlarge effective dataset	vision, audio, and now synthetic data for LLMs
Ensembling	average independent errors away	tabular competitions, and LLM self-consistency

9. Validation strategy

- **Hold-out** — simple, high variance on small data.
- **k-fold CV** — every point is validated once; $k=5$ or 10 . Expensive but honest.
- **Stratified k-fold** — preserve class ratios. Default for classification.
- **Group k-fold** — when rows share an entity (patient, customer, session), keep the group in one fold or you leak.
- **Time-series split** — always train on the past, validate on the future. Never shuffle time.
- **Nested CV** — outer loop estimates performance, inner loop tunes. The only unbiased way to report a score for a tuned model. Rarely done; frequently should be.

10. The test set is sacred

Every time you look at the test score and change something, you fit to it a little. After twenty looks it's just another validation set and your reported number is fiction. Lock it. Touch it once.

This is the exact pathology behind benchmark contamination in LLMs (Module 15): the field collectively ran thousands of experiments against MMLU, and MMLU stopped measuring what it claimed to.

11. Distribution shift

- **Covariate shift** — $P(x)$ changes, $P(y|x)$ stable (new customer segment).
- **Label shift** — $P(y)$ changes (fraud rate triples).
- **Concept drift** — $P(y|x)$ changes (the relationship itself moved — a pandemic, a pricing change).

Detect with: PSI / KS tests on input distributions, monitored prediction distributions, and delayed-label performance tracking. Respond with retraining cadence, not heroics.

12. Double descent (why the old textbook is incomplete)

Classical wisdom: error follows a U-curve as capacity grows. Empirically, once you cross the *interpolation threshold* (enough parameters to fit the training data perfectly), test error can **fall again**. Very overparameterised networks generalise well when trained with SGD-style implicit regularisation.

This is the theoretical permission slip for everything in Modules 02–04: yes, a model with a trillion parameters trained to near-zero training loss can still generalise.

Practice

1. Implement gradient descent for logistic regression; verify your analytic gradient against a numerical one $((J(\theta+\epsilon) - J(\theta-\epsilon)) / 2\epsilon)$. This gradient-check habit will save you in Module 02.
2. Plot learning curves (train & val error vs. training-set size) and classify the regime as bias- or variance-limited.
3. Deliberately leak a feature, observe the fantasy score, then remove it.